Labo 5 – Datavisualisatie I

Vertel het verhaal met plotly

Cursustekst bij het notebook `lab5_data_visualization_I.py`

dr. Maarten Slembrouck

Waarover gaat dit labo?

In twee labo's maakte je van rommelige exports nette, gecombineerde tabellen. Maar een coach leest geen tabel met 111 rijen. Een goede grafiek toont in één oogopslag wat in de tabel verborgen zit: wie het snelst is, wie vooruitgaat, hoe zwaar een training was.

Na dit labo kun je een grafiektype kiezen dat past bij een vraag, de standaardgrafieken maken met `plotly.express`, extra informatie tonen met kleur, facetten en grootte, en grafieken interactief maken in marimo.

Alle grafieken in deze tekst zijn gemaakt met de code die erboven staat, op de data van de club. In het notebook zijn ze interactief: je kunt inzoomen en met de muis over een punt gaan voor meer informatie.

1 De data voor dit labo

Het notebook begint met één cel die de volledige opkuis van labo 3 en 4 herhaalt. Daarna heb je twee tabellen:

- `combined`: één rij per atleet per testmoment, met positie, geslacht, gewicht, sprinttijden, spronghoogte en Yo-Yo-niveau;
- `hr_zoned`: de training per seconde, met hartslag, snelheid, naam en hartslagzone.

```
combined.shape, hr_zoned.shape
```

uitvoer

```
((111, 15), (12000, 11))
```

Beide zijn **lang**: één rij per meting. Dat is precies de vorm die `plotly.express` verwacht (zie labo 4).

Daarnaast legt het notebook vooraf één kleur per positie vast, in een dictionary. Zo is "Forward" in elke grafiek dezelfde kleur, en moet de lezer de legende maar één keer leren.

```
POSITION_COLORS = {
    "Forward": "#2a78d6",
    "Midfielder": "#eb6834",
    "Defender": "#1baf7a",
    "Goalkeeper": "#eda100",
}
```

Een kleur schrijf je als *hexcode*: een hekje gevolgd door zes tekens, twee voor rood, twee voor groen en twee voor blauw.

2 Principes: de juiste grafiek voor de juiste vraag

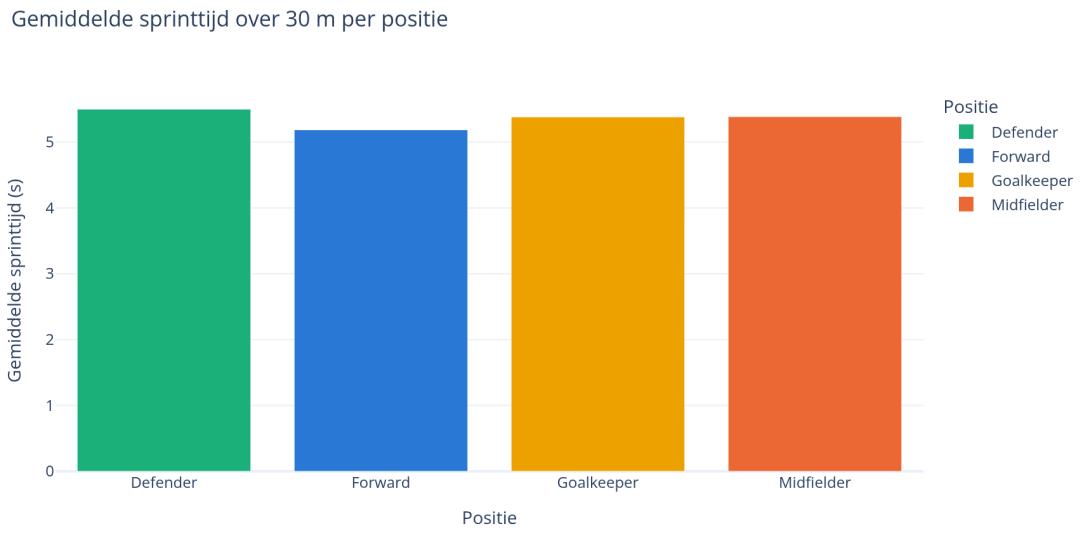
Elke grafiek beantwoordt een vraag. Begin daarom bij de vraag, niet bij het grafiektype dat je toevallig kent.

Soort vraag	Voorbeeld	Geschikte grafiek
Verdeling	Hoe zijn de sprinttijden gespreid?	histogram, boxplot
Vergelijking	Welke positie springt gemiddeld het hoogst?	staafdiagram, boxplot
Verband	Betekent een snellere sprint een hogere sprong?	spreidingsdiagram
Evolutie	Stijgt de hartslag van deze atleet?	lijngrafiek

2.1 Eerlijke assen

Een grafiek kan even goed misleiden als informeren. De bekendste valkuil is een **afgeknotte as**. Hieronder twee keer dezelfde gemiddelden. In de eerste grafiek begint de y-as bij 0:

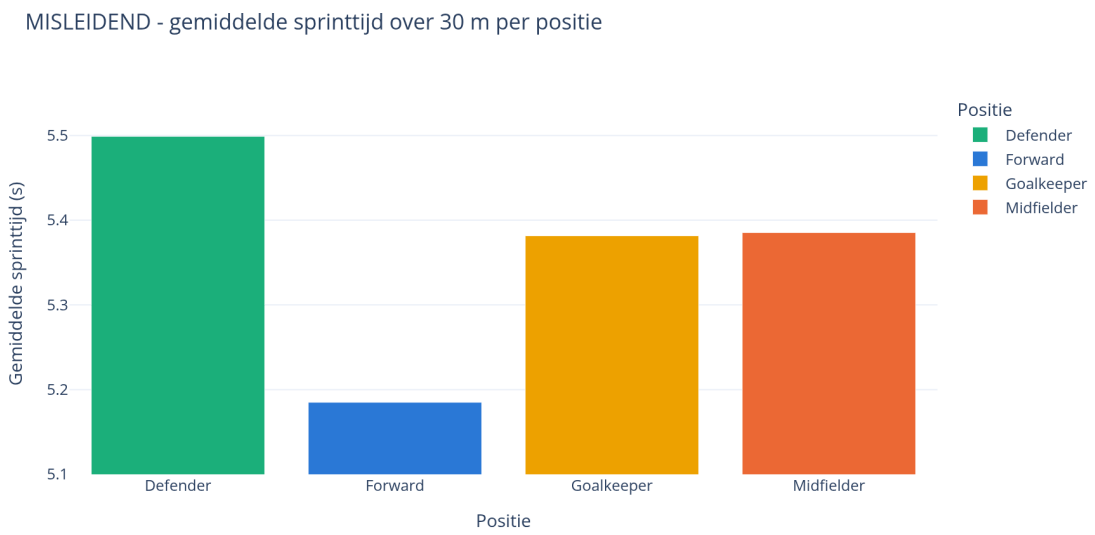
```
sprint_means = combined.groupby("position", as_index=False)["sprint_30m_s"].mean()
fig = px.bar(
    sprint_means, x="position", y="sprint_30m_s",
    color="position", color_discrete_map=POSITION_COLORS,
    title="Gemiddelde sprinttijd over 30 m per positie",
    labels={"sprint_30m_s": "Gemiddelde sprinttijd (s)", "position": "Positie"},
)
fig.update_yaxes(rangemode="tozero")
```



`as_index=False` zorgt ervoor dat `position` een gewone kolom blijft (en geen index wordt), zodat je ze als `x` kunt gebruiken. In de tweede grafiek begint de y-as net onder de kleinste waarde:

```
fig = px.bar(
    sprint_means, x="position", y="sprint_30m_s",
    color="position", color_discrete_map=POSITION_COLORS,
    title="MISLEIDEND - gemiddelde sprinttijd over 30 m per positie",
```

```
    labels={"sprint_30m_s": "Gemiddelde sprinttijd (s)", "position": "Positie"},
)
fig.update_yaxes(range=[5.1, 5.55])
```



Dezelfde kleine verschillen (hooguit 0,3 seconden) lijken nu enorm. Een staaf toont een grootte met haar *lengte*; dan moet die lengte bij nul beginnen. Om dezelfde reden gebruik je nooit twee verschillende y-assen in één grafiek: de lezer kan de twee schalen niet vergelijken.

3 Grafieken maken met `plotly.express`

`plotly.express` is een bibliotheek om met weinig code interactieve grafieken te maken. Je importeert ze als `px`:

```
import plotly.express as px
```

Elke functie volgt hetzelfde patroon:

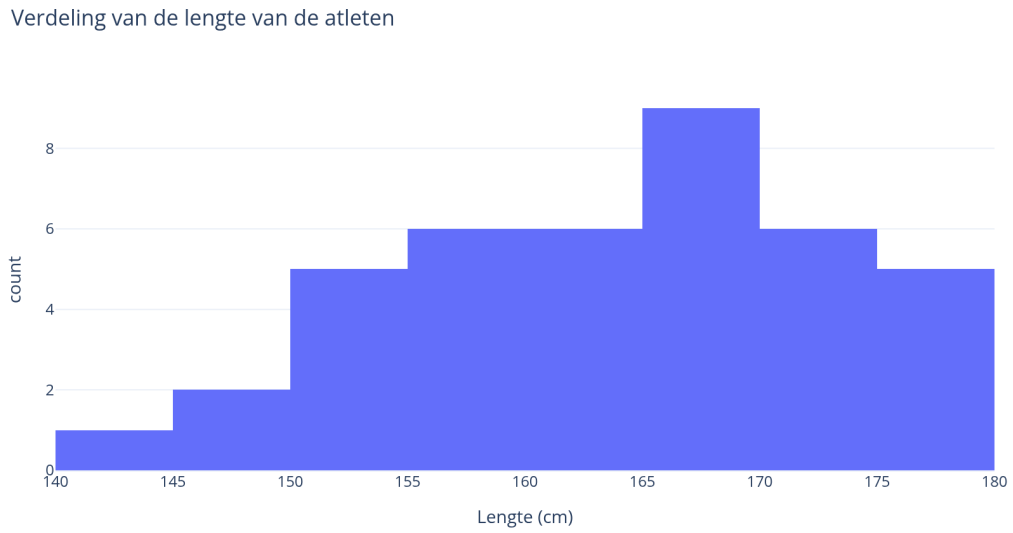
```
fig = px.SOORT(tabel, x="kolom", y="kolom", color="kolom",
               title="...", labels={"kolom": "Leesbare naam"})
```

- de **tabel** komt eerst; daarna verwijs je naar kolommen met hun naam, als tekst;
- `title` is de titel boven de grafiek;
- `labels` is een dictionary die kolomnamen vertaalt naar leesbare aslabels, mét eenheid: “Sprinttijd (s)” in plaats van `sprint_30m_s`;
- het resultaat is een **figuur**-object. Zet je de variabele als laatste regel in een cel, dan toont marimo de grafiek.

3.1 Histogram: hoe is één variabele verdeeld?

Een histogram verdeelt de waarden in vakjes en toont hoeveel waarden in elk vakje vallen. `nbins` kiest het (ongeveer) aantal vakjes.

```
fig = px.histogram(
    athletes, x="height_cm", nbins=15,
    title="Verdeling van de lengte van de atleten",
    labels={"height_cm": "Lengte (cm)"},
)
```



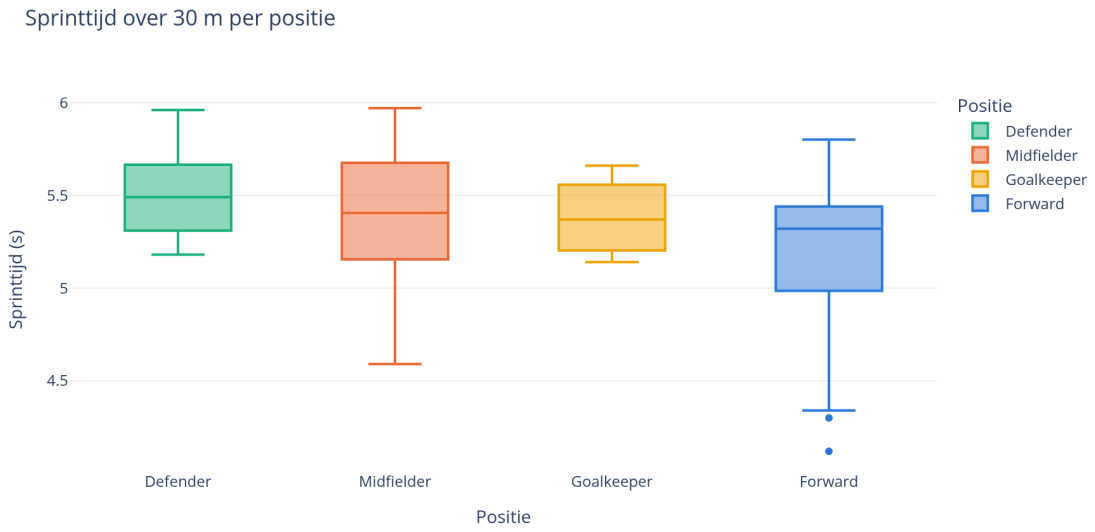
Een histogram heeft maar één kolom nodig (`x`); de y-as is het aantal atleten per vakje, dat telt plotly zelf. Speel met `nbins`: te weinig vakjes verbergen details, te veel maken de grafiek rommelig.

3.2 Boxplot: verdelingen vergelijken

Een boxplot vat een verdeling samen in vijf getallen. Zo lees je hem:

- de **lijn in de doos** is de mediaan: de helft van de waarden ligt eronder, de helft erboven;
- de **doos** loopt van het eerste tot het derde kwartiel: de middelste helft van de waarden;
- de **snorharen** (*whiskers*) lopen tot de laagste en hoogste waarde die niet uitzonderlijk ver van de doos liggen;
- **losse punten** zijn uitschieters.

```
fig = px.box(
    combined, x="position", y="sprint_30m_s",
    color="position", color_discrete_map=POSITION_COLORS,
    title="Sprinttijd over 30 m per positie",
    labels={"sprint_30m_s": "Sprinttijd (s)", "position": "Positie"},
    hover_data=["name"],
)
```

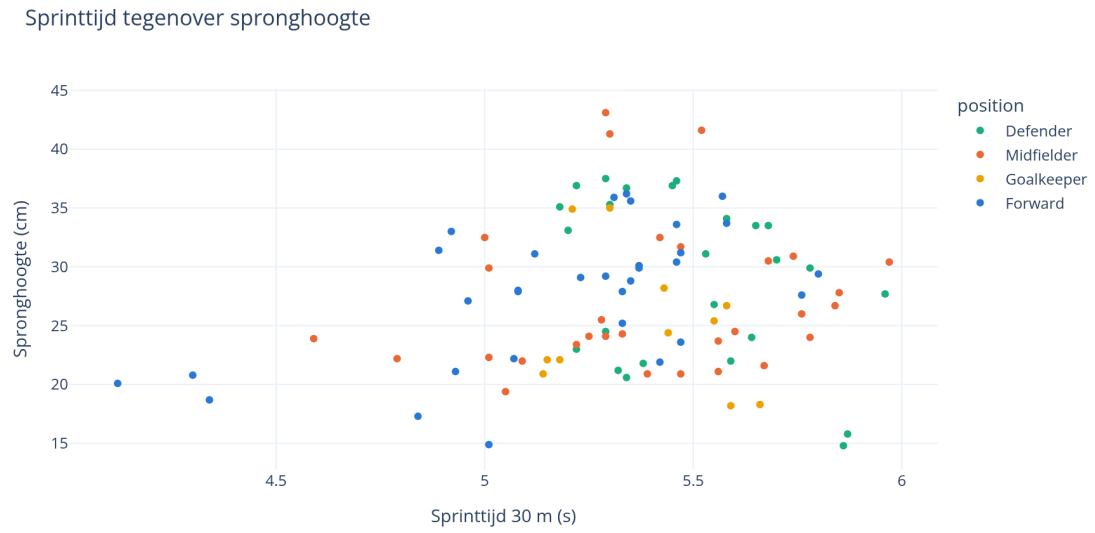


`hover_data=["name"]` voegt de naam toe aan het informatievenster dat verschijnt als je met de muis over een punt gaat. Zo zie je in het notebook meteen wie die uitschieter is.

3.3 Spreidingsdiagram: is er een verband?

In een spreidingsdiagram is elke rij een punt, met de ene variabele op de x-as en de andere op de y-as.

```
fig = px.scatter(
    combined, x="sprint_30m_s", y="cmj_height_cm",
    color="position", color_discrete_map=POSITION_COLORS,
    title="Sprinttijd tegenover spronghoogte",
    labels={"sprint_30m_s": "Sprinttijd 30 m (s)", "cmj_height_cm": "Spronghoogte
    ↪  (cm)"},
    hover_data=["name", "test_moment"],
)
```

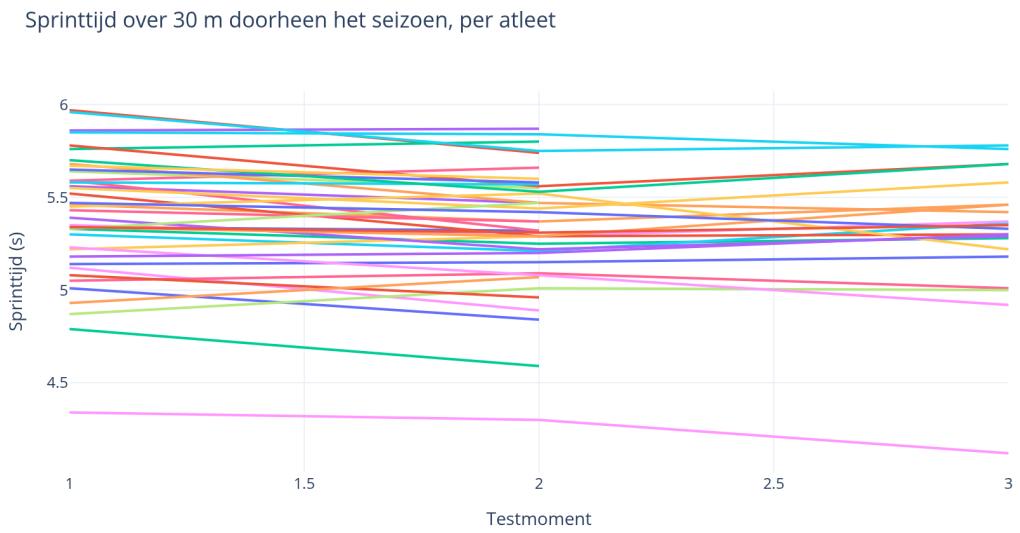


De punten vormen een brede wolk zonder duidelijke richting. Als er een sterk verband was, zouden ze rond een schuine lijn liggen: dalend als snellere sprinters (een lagere tijd) ook hoger springen. Hier is dat niet zo: in deze data zegt de sprinttijd weinig over de spronghoogte. Ook “geen duidelijk verband” is een antwoord op de vraag van de coach. En zie je wel een verband, onthoud dan dat dat nog geen bewijs is dat het ene het andere veroorzaakt.

3.4 Lijngrafiek: hoe verandert iets?

Een lijngrafiek verbindt opeenvolgende punten. Daarvoor moet de data **gesorteerd** zijn op de x-as, anders springt de lijn heen en weer. Met `color="athlete_id"` krijgt elke atleet een eigen lijn:

```
fig = px.line(
    combined.sort_values("test_moment"), x="test_moment", y="sprint_30m_s",
    color="athlete_id",
    title="Sprinttijd over 30 m doorheen het seizoen, per atleet",
    labels={"sprint_30m_s": "Sprinttijd (s)", "test_moment": "Testmoment"},
)
fig.update_layout(showlegend=False)
```



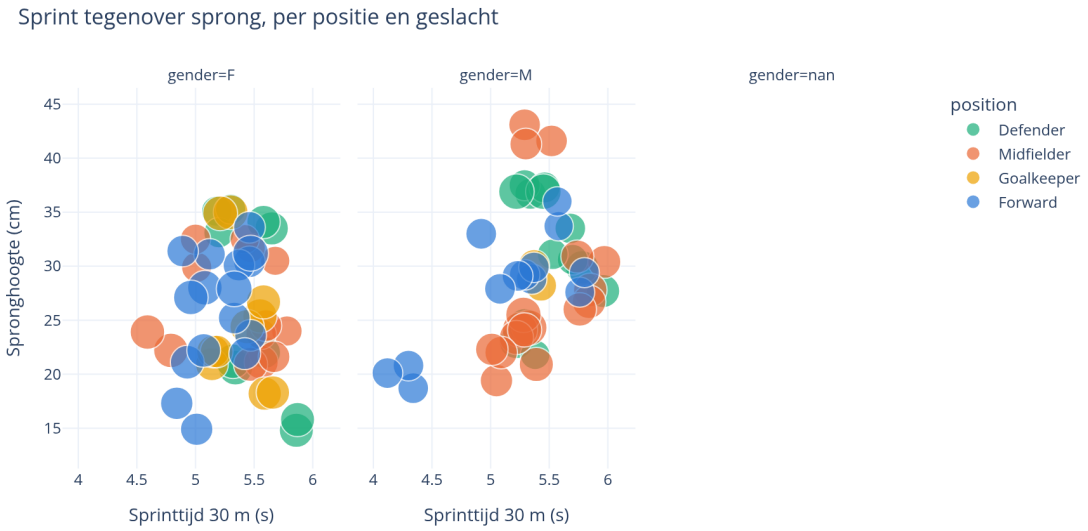
Met `update_layout(showlegend=False)` verberg je de legende: met 42 atleten zou die meer plaats innemen dan de grafiek zelf. Dit werkt zonder `pivot_table`, omdat `combined` al lang is.

4 Meer dimensies: kleur, facetten en grootte

Een spreidingsdiagram toont twee variabelen. Met drie extra argumenten kun je er meer tonen:

- `color="kolom"`: een categorie als kleur;
- `facet_col="kolom"`: één kleine grafiek per waarde, naast elkaar, met dezelfde assen;
- `size="kolom"`: een getal als grootte van het punt.

```
fig = px.scatter(
    combined, x="sprint_30m_s", y="cmj_height_cm",
    color="position", color_discrete_map=POSITION_COLORS,
    facet_col="gender", size="weight_kg",
    title="Sprint tegenover sprong, per positie en geslacht",
    labels={"sprint_30m_s": "Sprinttijd 30 m (s)", "cmj_height_cm": "Spronghoogte
    ↪  (cm)"},
)
```



Vijf variabelen in één grafiek: sprinttijd, spronghoogte, positie, geslacht en gewicht. Merk het derde, lege paneel `gender=nan` op: bij een paar rijen ontbreekt het geslacht, en plotly maakt daar een eigen paneel voor. Wil je dat niet, laat die rijen dan eerst weg met `combined.dropna(subset=["gender"])`, en vertaal de paneeltitels met `labels={"gender": "Geslacht"}`. Facetten zijn vaak duidelijker dan alles in één grafiek te proppen, omdat je de panelen naast elkaar kunt vergelijken. Maar meer is niet altijd beter: voeg een dimensie alleen toe als ze iets bijdraagt aan de vraag.

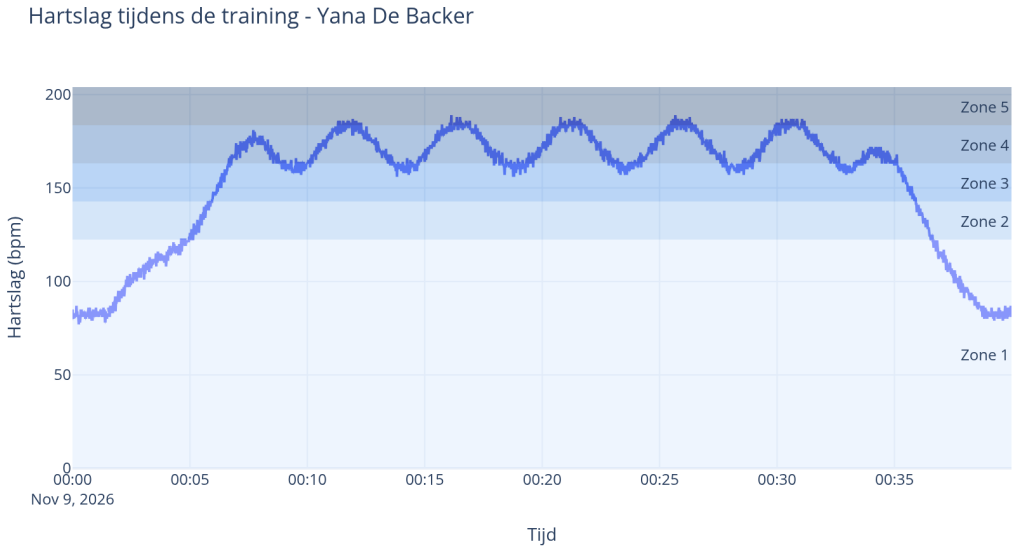
5 De hartslag als tijdreeks, met zones

Een lijngrafiek van de hartslag wordt veel leesbaarder als je de hartslagzones als gekleurde banden op de achtergrond toont. Daarvoor dient `add_hrect`: een horizontale rechthoek tussen twee waarden op de y-as.

```
ZONE_COLORS = {"Zone 1": "#cde2fb", "Zone 2": "#86b6ef", "Zone 3": "#3987e5",
               "Zone 4": "#1c5cab", "Zone 5": "#0d366b"}
one_athlete_hr = hr_zoned.loc[hr_zoned["athlete_id"] ==
↪  "A001"].sort_values("timestamp")
hr_max = one_athlete_hr["hr_max"].iloc[0]

fig = px.line(
    one_athlete_hr, x="timestamp", y="heart_rate_bpm",
    title=f"Hartslag tijdens de training - {one_athlete_hr['name'].iloc[0]}",
    labels={"heart_rate_bpm": "Hartslag (bpm)", "timestamp": "Tijd"},
)
zone_edges_pct = [0, 60, 70, 80, 90, 100]
```

```
zone_names = ["Zone 1", "Zone 2", "Zone 3", "Zone 4", "Zone 5"]
for lo, hi, zone in zip(zone_edges_pct[:-1], zone_edges_pct[1:], zone_names):
    fig.add_hrect(y0=lo / 100 * hr_max, y1=hi / 100 * hr_max,
                  fillcolor=ZONE_COLORS[zone], opacity=0.35, line_width=0,
                  annotation_text=zone, annotation_position="right")
fig
```



De lus verdient wat uitleg:

- `zone_edges_pct[:-1]` is de lijst zonder het laatste element (0, 60, 70, 80, 90): de ondergrenzen. `zone_edges_pct[1:]` is de lijst zonder het eerste element (60, 70, 80, 90, 100): de bovengrenzen.
- `zip(...)` loopt door drie lijsten tegelijk en geeft telkens één element van elke lijst: eerst `(0, 60, "Zone 1")`, dan `(60, 70, "Zone 2")`, enzovoort.
- `lo / 100 * hr_max` zet een percentage om naar een hartslag in slagen per minuut.

In het notebook gebruik je `_lo` in plaats van `lo`: in marimo blijft een naam die met `_` begint binnen de cel, zodat hij niet botst met variabelen in andere cellen.

6 Interactiviteit in marimo

6.1 UI-elementen en `.value`

marimo heeft **UI-elementen**: keuzemenu's, schuifbalken, tekstvakken. Een UI-element is een gewone Python-variabele. Het belangrijkste kenmerk is `.value`: de waarde die de gebruiker op dit moment gekozen heeft.

```
athlete_picker = mo.ui.dropdown(
    options=sorted(hr_zoned["name"].unique()),
    value=sorted(hr_zoned["name"].unique())[0],
    label="Atleet",
)
time_window = mo.ui.range_slider(
```

```
    start=0, stop=2400, step=60, value=(0, 2400),
    label="Tijdvenster (s)", show_value=True,
)
mo.hstack([athlete_picker, time_window])
```

- `options` zijn de keuzes; `value` is de beginkeuze. `.unique()` geeft elke naam één keer, `sorted` zet ze alfabetisch.
- een `range_slider` heeft twee handvatten; zijn `.value` is een tuple, bv. `(0, 2400)`.
- `mo.hstack([...])` zet elementen naast elkaar, `mo.vstack([...])` onder elkaar.

In een *andere* cel gebruik je de gekozen waarden om de data te filteren en de grafiek te maken:

```
selected_hr = hr_zoned.loc[
    (hr_zoned["name"] == athlete_picker.value)
    & hr_zoned["time_s"].between(*time_window.value)
].sort_values("timestamp")

reactive_hr_chart = px.line(selected_hr, x="timestamp", y="heart_rate_bpm",
                            title=f"Hartslag - {athlete_picker.value}")
reactive_hr_chart
```

`.between(a, b)` geeft een masker dat `True` is voor waarden van `a` tot en met `b`. Het sterretje in `between(*time_window.value)` *pakt de tuple uit*: `between(*(0, 2400))` is hetzelfde als `between(0, 2400)`.

Waarom twee cellen?

marimo voert een cel opnieuw uit als een variabele waarvan ze afhangt verandert. Als je het keuzemenu verzet, verandert `athlete_picker.value`, en voert marimo elke cel die `athlete_picker` gebruikt opnieuw uit. De cel die het element *maakt* mag de `.value` niet zelf lezen: dan zou het element bij elke keuze opnieuw gemaakt worden. Daarom: **maak een UI-element in de ene cel, lees zijn waarde in een andere**.

6.2 Selecteren in de grafiek (optioneel)

`mo.ui.plotly(figuur)` gaat nog een stap verder: als je in de grafiek punten selecteert met de box- of lassotool, zitten die punten in `.value`. Met `pd.DataFrame(selectable_scatter.value)` krijg je ze als tabel.

7 Het minidashboard

De opdracht van vandaag: een klein dashboard voor de coach, met **drie grafieken** en **één keuzemenu** waarop minstens één grafiek reageert. Het keuzemenu (`dashboard_selector`, een keuze uit de posities) staat al klaar in het notebook. Je maakt een lijst `dashboard_charts` met drie figuren.

Een paar tips:

- **Hergebruik.** Het doel is combineren, niet nieuwe grafiektypes leren. Alles uit dit labo mag terugkomen.
- **Eén vraag per grafiek.** Schrijf eerst voor elke grafiek op welke vraag van de coach ze beantwoordt, en kies dan het type (tabel in sectie 2).
- **Filter met de selectie.** Filter de data met `dashboard_selector.value` voor je de grafiek

maakt, zoals bij de hartslaggrafiek hierboven.

- **Afwerking.** Elke grafiek heeft een titel en leesbare aslabels met eenheden, en gebruikt `POSITION_COLORS` waar posities voorkomen.
- **Schikken.** Met `mo.vstack` en `mo.hstack` zet je het keuzemenu en de grafieken netjes onder en naast elkaar.

8 Samenvatting

Code	Wat het doet
`px.histogram(df, x=, nbins=)`	verdeling van één variabele
`px.box(df, x=, y=)`	verdelingen vergelijken per groep
`px.scatter(df, x=, y=)`	verband tussen twee variabelen
`px.line(df.sort_values(x), x=, y=)`	evolutie; eerst sorteren
`px.bar(df, x=, y=)`	waarden vergelijken; as vanaf 0
`title=`, `labels={...}`, `hover_data=[...]`	titel, leesbare aslabels, extra info bij hover
`color=`, `color_discrete_map=`	categorie als kleur, met vaste kleuren
`facet_col=`, `size=`	panelen per waarde, getal als puntgrootte
`fig.update_yaxes(rangemode="tozero")`	y-as laten beginnen bij 0
`fig.add_hrect(y0=, y1=, ...)`	horizontale band, bv. een hartslagzone
`mo.ui.dropdown`, `mo.ui.range_slider`	keuzemenu, schuifbalk met twee handvatten
`element.value`	de huidige keuze van de gebruiker
`mo.hstack`, `mo.vstack`	elementen naast of onder elkaar zetten

Veelgemaakte fouten

- Een staafdiagram met een y-as die niet bij 0 begint.
- Geen titel of geen eenheden op de assen.
- Een lijngrafiek op data die niet gesorteerd is op de x-as.
- Een UI-element maken en zijn `.value` in dezelfde cel lezen.
- Elke grafiek eigen kleuren laten kiezen, zodat “Forward” in de ene grafiek blauw is en in de andere oranje.

Begrippen

figuur het object dat een `px`-functie teruggeeft; marimo toont het als grafiek.

mediaan middelste waarde: de helft ligt eronder, de helft erboven.

kwartiel grens van de laagste of hoogste 25 % van de waarden.

uitschieter waarde die ver van de andere ligt.

facet klein paneel met dezelfde assen, één per waarde van een kolom.

UI-element keuzemenu, schuifbalk of ander element waarmee de gebruiker iets kiest.